

Estructuras de Datos

Clase 14 – Árboles Generales (segunda parte)



Dr. Sergio A. Gómez
<http://cs.uns.edu.ar/~sag>



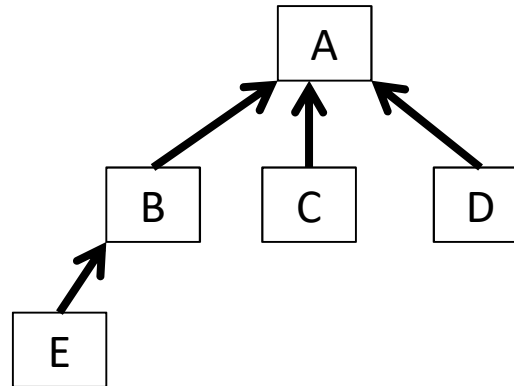
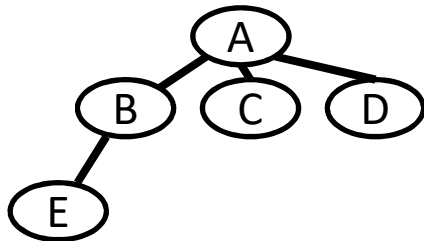
Departamento de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur
Bahía Blanca, Argentina

Representaciones de árboles

- Del padre
- Lista de hijos
- Goodrich & Tamassia: Del padre + Lista de hijos
- Hijo extremo izquierdo – hermano derecho
- Variantes de las de más arriba usando arreglos con cursores (de interés histórico o para modelar árboles en almacenamiento externo)

Representación del padre

- Cada nodo conoce su rótulo y a su padre.



- Aplicación: Directorio padre en sistemas de archivos
- El directorio padre se denota con .. (dos puntos consecutivos).

El directorio corriente es c:\windows
El comando `cd..` lleva al directorio padre.

```
C:\Windows\system32\cmd.exe - dir /ad /p
C:\Windows>dir /ad /p
Volume in drive C has no label.
Volume Serial Number is 0A37-3572

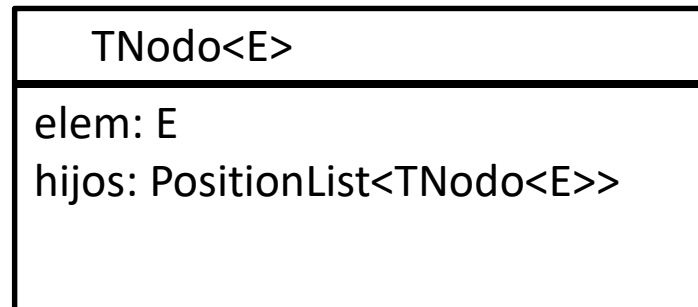
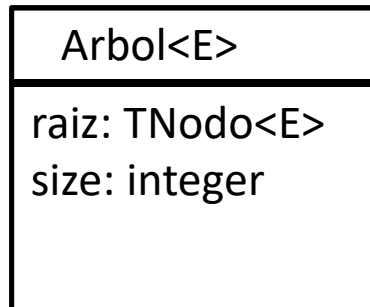
Directory of C:\Windows

27/02/2013  03:14 p.m.    <DIR>
27/02/2013  03:14 p.m.    <DIR>
14/07/2009  02:32 a.m.    <DIR>
14/07/2009  12:20 a.m.    <DIR>
19/03/2013  08:30 a.m.    <DIR>
               .
               ..
               addins
               AppCompat
               AppPatch
```

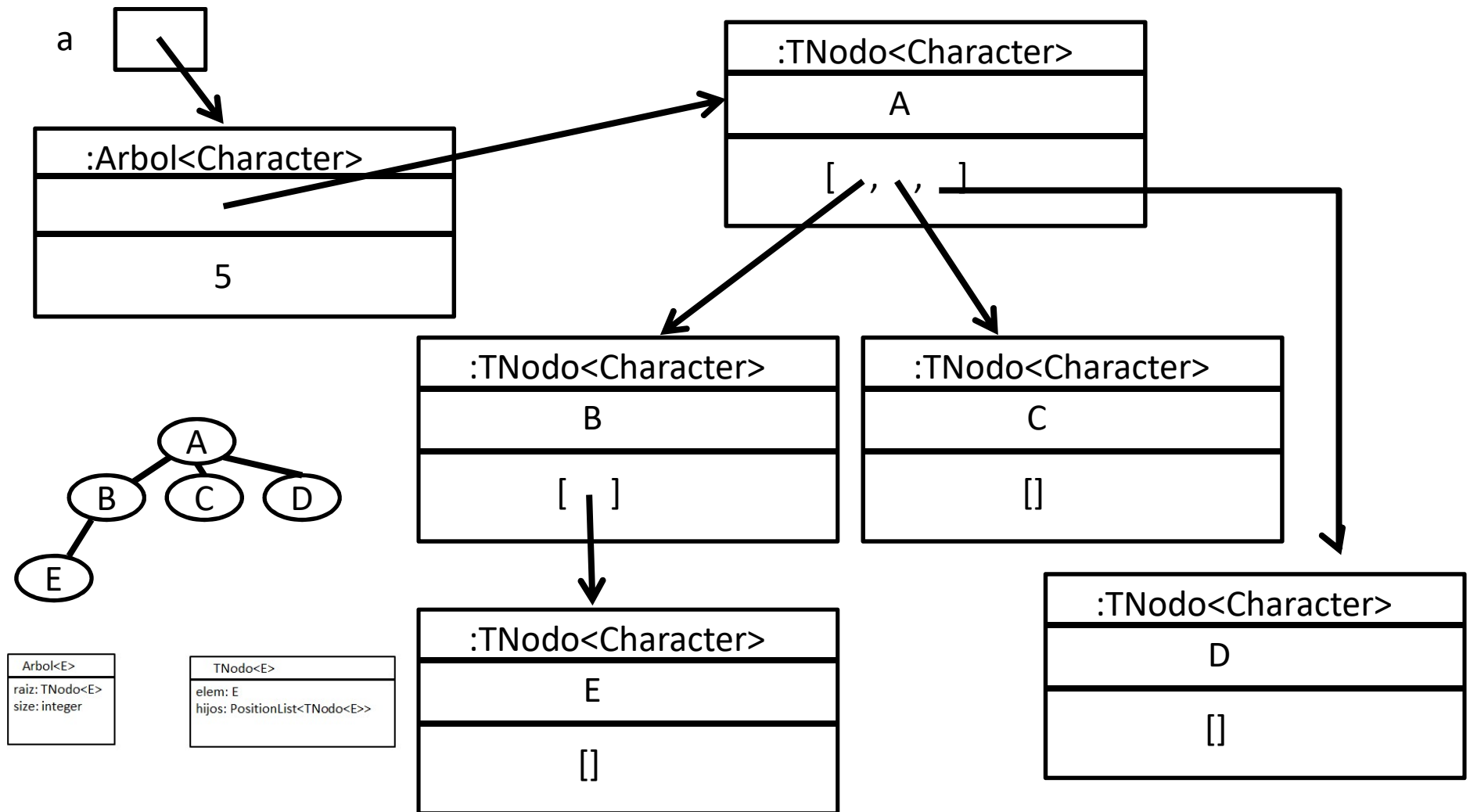
“..” es la referencia al directorio padre

Representación de lista de hijos

- Cada nodo conoce su rótulo (elemento) y la lista de sus nodos hijos
- El árbol conoce el nodo raíz del árbol



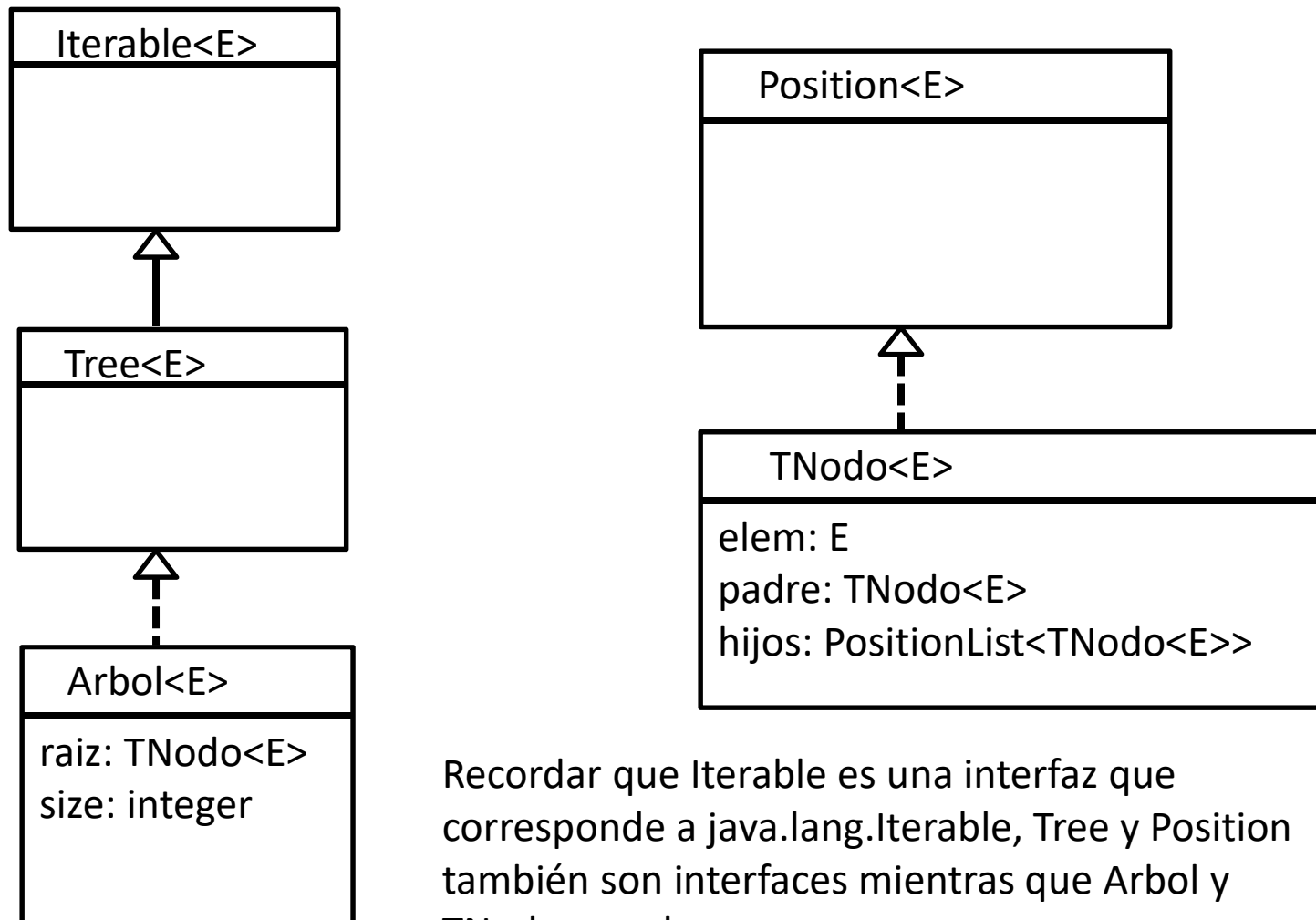
Representación de lista de hijos



Representación en GT: Colección de hijos

- GT = Lista de hijos + padre
- El árbol conoce la raíz
- Cada nodo conoce el rótulo (elemento), la lista de nodos hijos y el nodo padre.
- (Discutiremos la estructura de datos y la implementación de algunas de las operaciones, el resto quedará a cargo del alumno.)

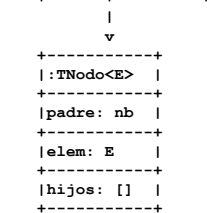
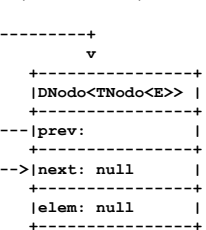
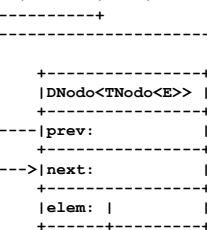
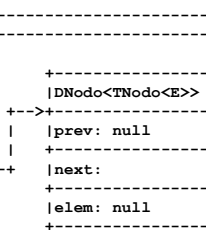
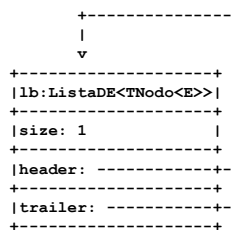
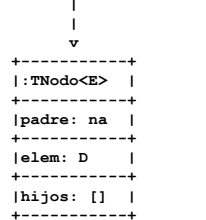
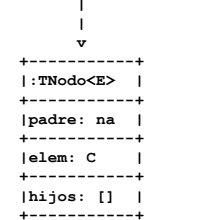
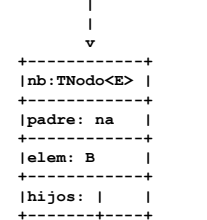
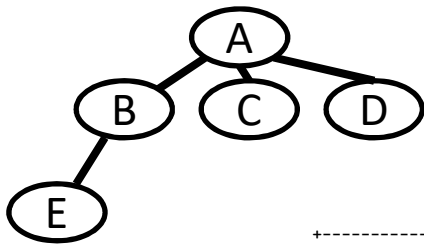
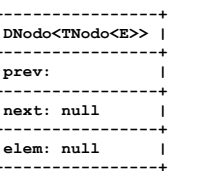
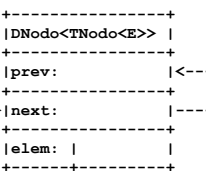
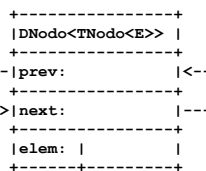
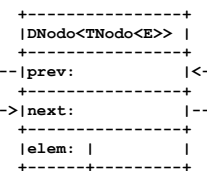
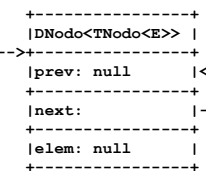
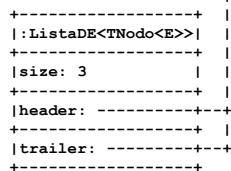
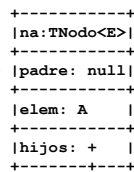
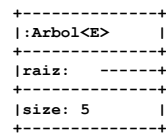
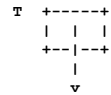
Representación de árboles en GT



Recordar que Iterable es una interfaz que corresponde a java.lang.Iterable, Tree y Position también son interfaces mientras que Arbol y TNode son clases.

Representación de árboles en GT: Diagrama de objetos

Tree<E> T



Abuso de notación:

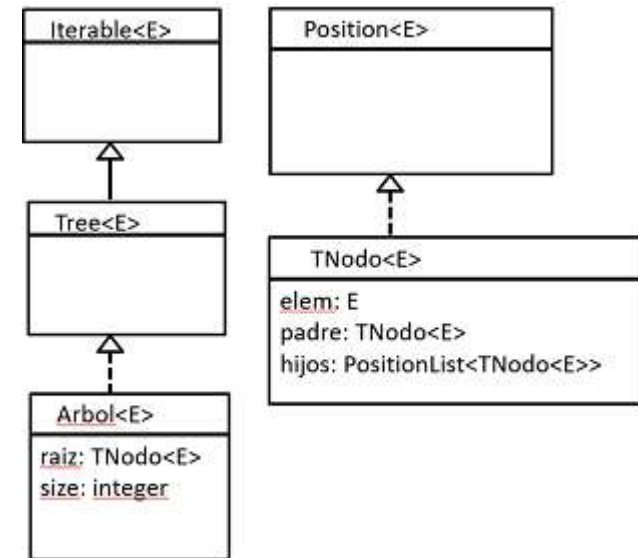
A las listas de hijos de C, D, E simplemente las graficamos como [] porque están vacías.

Representación de árboles en GT: Clase TNode

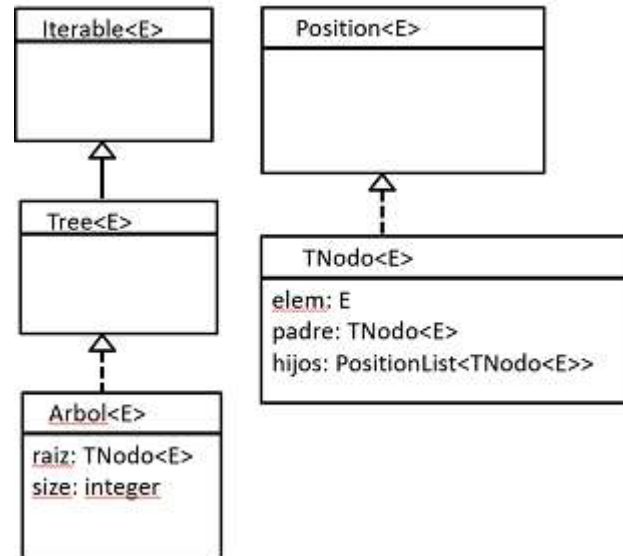
```
public class TNode<E> implements Position<E>
{
    private E elemento;
    private TNode<E> padre;
    private PositionList<TNode<E>> hijos;

    public TNode( E elemento, TNode<E> padre ) {
        this.elemento = elemento;
        this.padre = padre;
        hijos = new DoubleLinkedList<TNode<E>>();
    }

    public TNode(E elemento ) { this( elemento, null); }
    public E element() { return elemento; }
    public PositionList<TNode<E>> getHijos() { return hijos; }
    public void setElemento( E elemento ) { this.elemento = elemento; }
    public TNode<E> getPadre() { return padre; }
    public void setPadre( TNode<E> padre ) { this.padre = padre; }
}
```



Representación de árboles en GT: constructor e isEmpty



```
public class Arbol<E> implements Tree<E>
{
```

```
    protected TNode<E> raiz;
    protected int size;
```

```
    public Arbol() { raiz = null; size = 0; }
```

```
    public boolean isEmpty() { return raiz == null; }
```

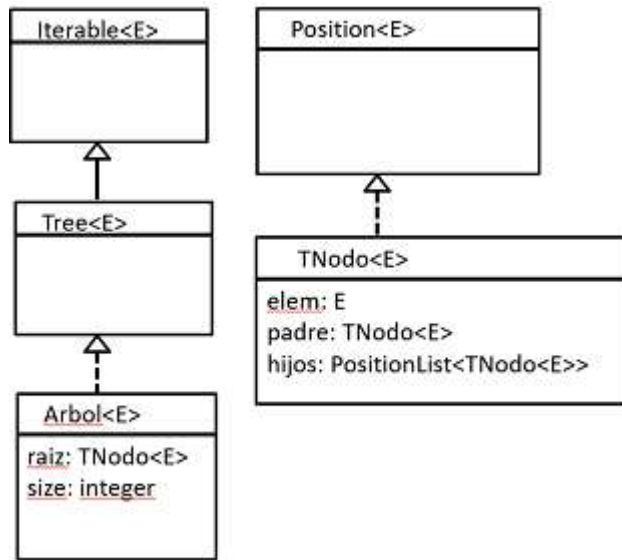
```
Tree<E> T = new Arbol<E>();
```

```

T  +-----+
   |   |   |
   +---+---+
       |
       v
+-----+
| :Arbol<E> |
+-----+
| raiz:  null |
+-----+
| size:  0   |
+-----+
    
```

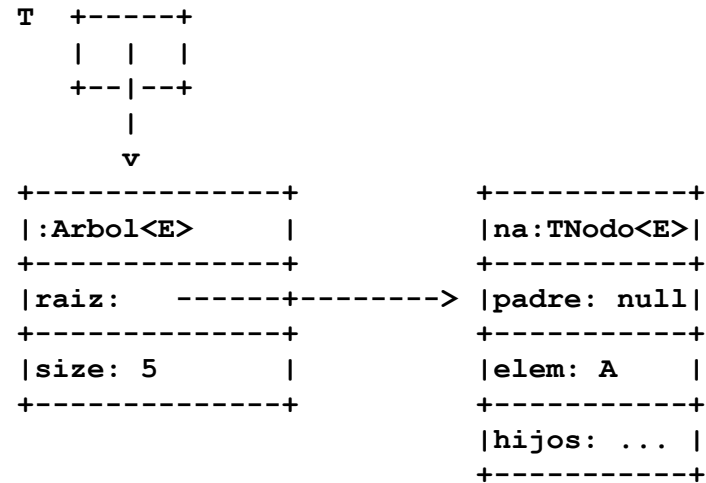
$T_{isEmpty}(n) = O(1)$

Representación de árboles en GT: raíz



```

Tree<E> T = ...
Position<E> raiz = T.root();
    
```

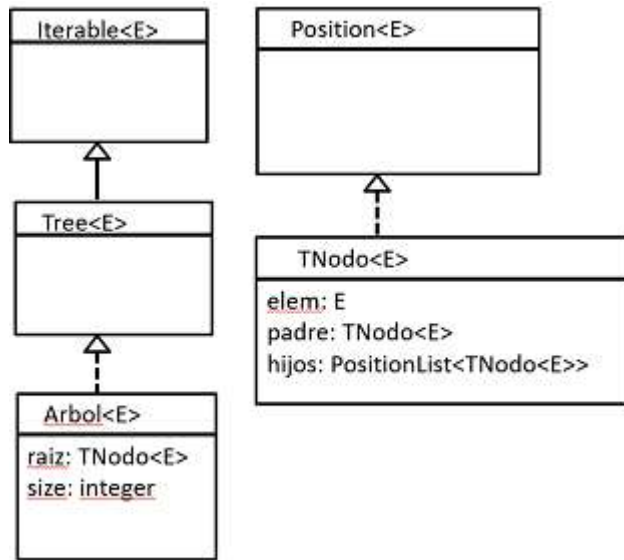


```

public Position<E> root() {
    if( isEmpty() )
        throw new EmptyTreeException( "Árbol vacío" );
    return raiz;
}
    
```

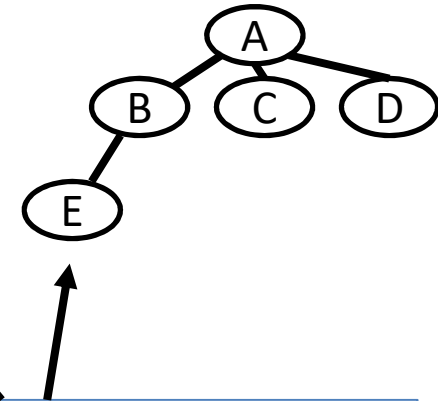
$$T_{\text{root}}(n) = O(1)$$

Representación de árboles en GT: ¿EsHoja?



```

+-----+
| :TNode<E> |
+-----+
|padre: nb |
+-----+
|elem: E   |
+-----+
|hijos: [] |
+-----+
    
```



La lista de hijos de un nodo hoja está vacía y la de un nodo interno no está vacía.

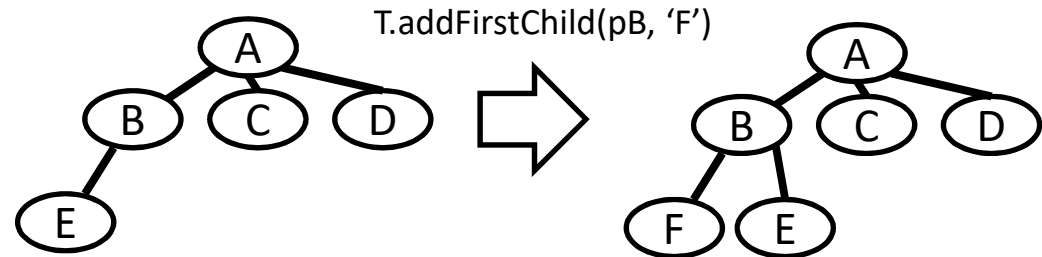
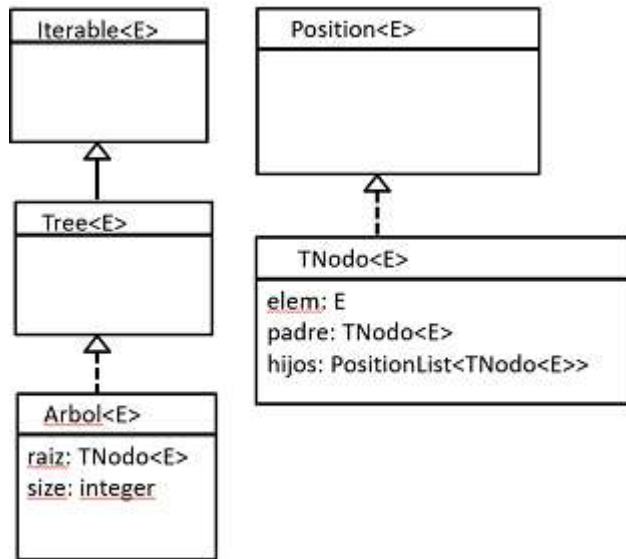
```

public boolean isExternal(Position<E> v) {
    TNode<E> nodo = checkPosition( v );
    return nodo.getHijos().isEmpty();
}
    
```

$$T_{\text{isExternal}}(n) = O(1)$$

Representación de árboles en GT:

Insertar primer hijo de p con rótulo e



Hay que castear p a TNode para poder tener acceso a sus campos, recuperar la lista de hijos de p, insertarle un nuevo nodo del árbol con rótulo e e incrementar el tamaño del árbol.

```

public Position<E> addFirstChild(Position<E> p, E e) {
    TNode<E> nodo = checkPosition(p);
    TNode<E> nuevo = new TNode<E>( e, nodo );
    nodo.getHijos().addFirst( nuevo );
    size++;
    return nuevo;
}

```

$T_{\text{addFirstChild}}(n) = O(1)$

Representación de árboles en GT:

Insertar un hijo e de p que con hermano derecho rb

```
import tda.lista.Position; // Necesito usar posiciones de lista por lista de hijos
public Position<E> addBefore(Position<E> p, Position<E> rb, E e) {
    TNode<E> n = checkPosition( p );
    TNode<E> hd = checkPosition( rb );
    TNode<E> nuevo = new TNode<E>( e, n );
    PositionList<TNode<E>> hijos = n.getHijos();
    // Buscar dónde está rb en la lista de hijos de p
    boolean encuentre = false;
    tda.lista.Position<TNode<E>> pp = hijos.first();
    while( pp != null && !encuentre )
        // Testeo si el elemento corriente de la lista de hijos de p es rb
        if( hd == pp.element() ) encuentre = true // Sí, es! => terminé el bucle
        else pp = (pp != hijos.last() ? hijos.next(pp) : null); // No es => avanzo
    if( !encuentre ) // => Hay un problema con los argumentos
        throw new InvalidPositionException( "p no es padre de rb" );
    hijos.addBefore( pp, nuevo ); // Inserto al nodo nuevo delante de rb
    size++; // Incremento el tamaño del árbol
    return nuevo; // Retorno el nodo creado
}
```

$T_{\text{addBefore}}(n) = O(\text{cantidad de hijos de } p)$

Árboles generales: Eliminar una hoja

```
import tdalista.Position; // Necesito usar posiciones de lista por lista de hijos
public void removeExternalNode (Position<E> p) {
    if( isEmpty() ) throw new InvalidPositionException( "Arbol vacío" );
    TNode<E> n = checkPosition( p );
    if( !n.getHijos().isEmpty() ) throw new InvalidPositionException( "p no es hoja" );
    if ( p == root() ) { raiz = null; size = 0; n.setElemento(null); return; } // Parche por caso no considerado
    TNode<E> padre = n.getParent(); // Obtengo el padre de n
    PositionList<TNode<E>> hijosPadre = padre.getHijos(); // Obtengo los hijos del padre de n
    boolean encuentre = false; // Hago una búsqueda lineal de n en la lista de hijos del padre de n
    tdalista.Position<TNode<E>> pp = null; // va a almacenar la posición de n en la lista de hijos de su padre
    Iterable<tdalista.Position<TNode<E>>> posiciones = hijosPadre.positions();
    Iterator<tdalista.Position<TNode<E>>> it = posiciones.iterator();
    while ( it.hasNext() && !encontre ) { // Para cada posición pp de la lista de hijos del padre de n
        pp = it.next(); // Pido la siguiente posición de nodo
        if ( pp.element() == n ) encuentre = true; // Testeo si pp es la posición de n
    }
    if( !encontre ) // No encontré a n entre los hijos de su padre: entro en pánico, no hago nada y aviso
        throw new InvalidPositionException( "p no aparece en la lista de hijos de su padre: ¡no eliminé!" );
    hijosPadre.remove( pp ); // La búsqueda fue exitosa: elimino a n de la lista de hijos de su padre.
    n.setElemento(null); // Puedo setear el rótulo en null para ayudar a checkPosition.
    size--; // Decremento el tamaño del árbol.
}
```

$T_{\text{removeExternalNode}}(n) = O(\text{cantidad de hijos del padre de } p) = O(\text{grado del árbol})$

Estructuras de datos - Dr. Sergio A. Gómez

Árboles generales: obtener los hijos de un nodo

```
public Iterable<Position<E>> children( Position<E> v )  
    throws InvalidPositionException {  
    TNode<E> p = checkPosition(v);  
    PositionList<Position<E>> lista =  
        new ListaDoblementeEnlazada<TNode<E>>();  
    for( TNode<E> n : p.getHijos() )  
        lista.addLast(n);  
    return lista;  
}
```

El tiempo de ejecución es del orden de la cantidad de hijos de v
La cantidad de hijos de v se puede acotar con el grado del árbol.
Grado del árbol: Máxima cantidad de hijos de un nodo.

Árboles generales: Iterador de posiciones

Construyo un iterador de posiciones del árbol recorriendo el árbol en preorden desde la raíz y encolando los nodos del árbol en una lista:

```
public Iterable<Position<E>> positions() {  
    PositionList<Position<E>> l = new ListaDoblementeEnzalada<Position<E>>();  
    // Si el árbol no está vacío, hago un recorrido preorden desde la raíz:  
    if( !isEmpty() ) pre(raiz, l);  
    return l;  
}
```

$T_{\text{positions}}(n) = O(n)$ si árbol this tiene n nodos

```
private void pre(TNodo<E> v, PositionList<Position<E>> l) {  
    l.addLast( v );           // La visita de v consiste de encolar v en l  
    for( TNodo<E> h : v.getHijos()) // para cada hijo h de v hacer  
        pre( h, l );          // preorden de h  
}
```

Nota: El tiempo de ejecución es lineal en la cantidad de nodos del árbol

Nota: pre es procedimiento y no función porque no tengo una operación que me permita concatenar dos listas en forma eficiente.

Nota: Se podría también hacer un iterador postorden, inorder o por niveles.

Árboles generales: Iterador de rótulos

Construyo un iterador de rótulos del árbol a partir del iterador de posiciones encolando los rótulos del árbol en una lista l:

```
public Iterator<E> iterator() {  
    PositionList<E> l = new ListaDoblementeEnlazada<E>();  
    for( Position<E> p : positions() )  
        l.addLast( p.element() );  
    return l.iterator();  
}
```

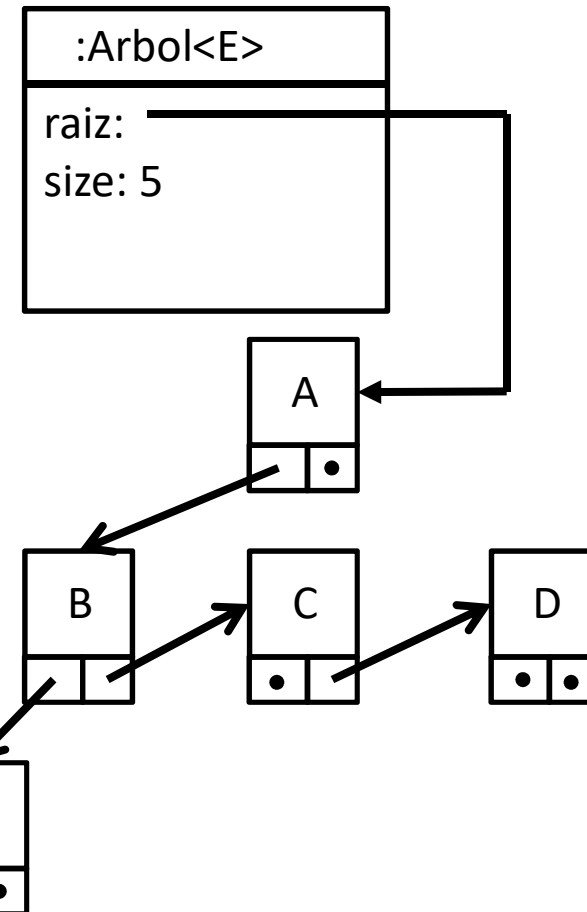
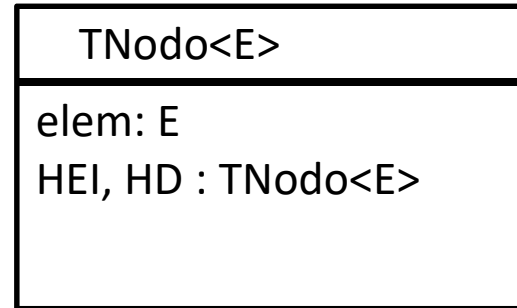
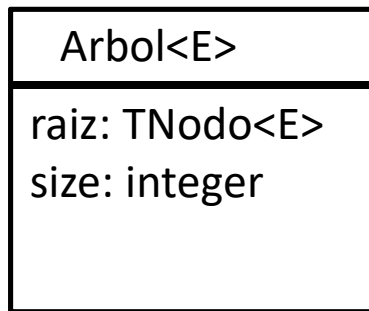
$T_{\text{iterator}}(n) = O(2n)$ si árbol this tiene n nodos

ya que crear la lista l toma $O(1)$, positions toma $O(n)$, el for-each vuelve a recorrer la lista retornada por positions (que mide n), l.addLast toma $O(1)$ (porque la lista es doblemente enlazada y tiene centinela final) y pedirle el iterador a la lista l toma $O(1)$.

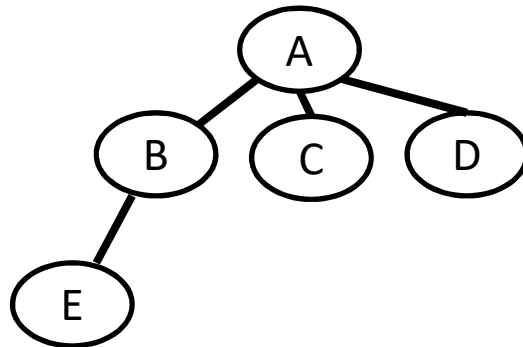
Otras representaciones:

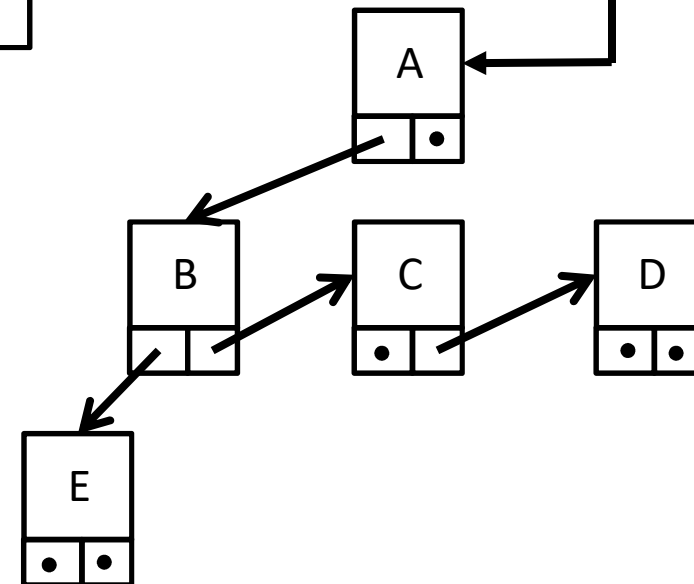
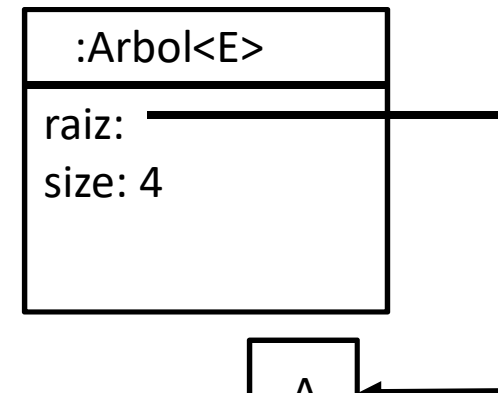
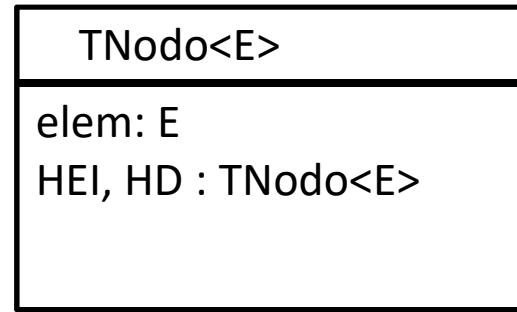
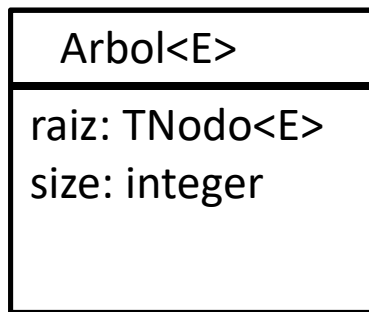
hijo extremo izquierdo-hermano derecho

Cada nodo conoce su rótulo y la identidad de su hijo extremo izquierdo y de su hermano derecho (se puede combinar con la representación del padre de ser necesario).



Ejemplo:





Recorrido preorden:

```

public void preorden() {
    pre( raiz );
}

```

// v representa la raíz del subárbol listado actualmente

```

private void pre( TNode<E> v ) {
    System.out.println( v.getElem() );
    TNode<E> h = v.getHEI();
    while( h != null ) {
        pre( h );
        h = h.getHD();
    }
}

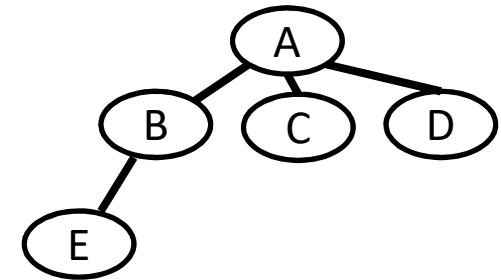
```

Representación con arreglos de HEI-HD

Esto es útil cuando el lenguaje no tiene soporte de memoria dinámica (e.g. ensamblador o quiero representar el árbol usando archivos binarios).

Representación: Tengo un arreglo de rótulos, un arreglo de padres, un arreglo de HEI, un arreglo de HD, cada nodo se representa con un cursor (una posición del arreglo), 0 indica el nodo nulo, la posición 0 del arreglo no se usa.

Anoto cuál es la posición del nodo raíz y el tamaño del árbol.



	0	1	2	3	4	5							
rótulos		A	B	C	D	E							
padres		0	1	1	1	2							
HEI		2	5	0	0	0							
HD		0	3	4	0	0							
raíz		1											
size		5											

Recorrido preorden

```
public void preorden() {  
    pre( raiz );  
}
```

// v representa el cursor de la raíz del subárbol listado actualmente

```
private void pre( int v ) {  
    System.out.println( rotulos[v] );  
    int h = HEI[v];    // h representa el hijo actual procesado  
    while( h != 0 ) {  // mientras h no sea nulo hacer  
        pre( h );      // preorden de h  
        h = HD[h];     // avanzar al siguiente hijo h (que es el hermano de h)  
    }  
}
```